Property Price Prediction Using Machine Learning (Data Science)

Submitted for the Mini Project

In

Department of Information Technology

by

Aditya Madhukar Gadiwan

Guided by

Mrs. Parvati Jadhav

SHRI GURU GOBIND SINGHJI INSTITUTE OF ENGINEERING AND TECHNOLOGY, NANDED

April 2025

Abstract

This project focuses on the prediction of property prices using machine learning techniques. A real-world dataset of home properties was collected from Kaggle in Excel format, containing attributes such as area type, availability, location, size, society, total square feet, number of bathrooms, balconies, and price. The project involved extensive data preprocessing including data cleaning, dimensionality reduction, outlier detection, feature engineering, and data visualization [4].

Only relevant features were retained: `location`, `total_sqft`, `bath`, `bhk`, and `price`. Three machine learning models were trained and evaluated – Linear Regression, Lasso Regression, and Decision Tree Regressor. Among these, Linear Regression showed the highest accuracy with an R2 score of 0.818.

The model was saved using Python's `pickle` module. To serve the model, a Flask web server was built with two key scripts: `server.py` for routing and handling API requests, and `util.py` for prediction logic. A JSON file (`columns.json`) was created to store feature metadata.

Testing and interaction were done using Postman, sending GET and POST requests to retrieve available locations and predicted prices. Finally, a web interface was developed using HTML, CSS, and JavaScript to allow users to input data and get real-time property price predictions.

Acknowledgements

I would like to thank Mrs. Parvati Jadhav, my project guide, for her continuous support, guidance, and encouragement throughout the course of this project. I also extend my gratitude to my peers and faculty of the Information Technology Department at SGGS for their insightful feedback and cooperation. Special thanks to Kaggle for providing open access to the dataset used.

Contents

Abstract **i**

Acknowledgements **ii**

List of Figures **iv**

List of Tables **v**

1 Introduction **1**
1.1 Project Overview 2

2 Dataset Description and Preprocessing **3**
2.1 Dataset Source 3
2.2 Data Cleaning and Processing 4

3 Model Building and Evaluation **7**

4 Web Application Development **9**
4.1 Flask Server 9
4.2 Testing with Postman 9
4.3 Frontend Interface 11

5 Experimental Result **12**

6 Conclusion and Future Work **15**

Bibliography **16**

List of Figures

2.1 Original dataset (Table 1) 4
2.2 Clean dataset (Table 2) 5
2.3 Outliers detection 6
2.4 Data cleaning 6
2.5 Statistical data 6

3.1 Accuracy comparision (Table 3) 7

4.1 Price prediction 10
4.2 Locations 10
4.3 Website Interface 11

5.1 Results 14

List of Tables

Table 1: Original Dataset Features .
Table 2: Sample Data After Cleaning .
Table 3: Accuracy Comparison Between Models .

Chapter 1

Introduction

The exponential growth of data in today's digital era has opened new frontiers for innovation, analysis, and problem-solving across industries. As a result, data science has emerged as one of the most transformative fields, enabling intelligent systems that can learn from data, make accurate predictions, and automate complex decision-making processes. This project leverages data science techniques to develop a robust and scalable solution that not only demonstrates model-building proficiency but also culminates in a real-time, interactive web application. The aim is to offer users an intelligent platform that interprets complex data and presents outcomes in a simplified, accessible manner.

The core objective of the project is to explore the full lifecycle of a machine learning project, from raw data to deployment. It begins with data collection and preprocessing, where null values, duplicates, outliers, and inconsistencies are addressed through rigorous cleaning techniques. Exploratory data analysis (EDA) is conducted to identify patterns, correlations, and feature importance. This step lays the foundation for selecting appropriate features that significantly impact the model's performance. Multiple algorithms such as Linear Regression, Decision Tree, and Lasso Regression are trained and evaluated to find the best fit for the given dataset. Model selection is guided by evaluation metrics like R-squared, Mean Squared Error (MSE), and cross-validation scores to ensure generalizability and accuracy.

One of the key highlights of this project is the development of a web application using modern web technologies. The application is designed to be lightweight, responsive, and easy to use, offering users the ability to interact with the model through a simple form-based interface. Behind the scenes, the machine learning model is served using a Flask API, which handles user inputs, performs backend computation, and returns predictions instantly. The interface ensures that non-technical users can also benefit

from the predictive power of the model without needing to understand the technical complexities involved.

In addition to technical implementation, this project emphasizes user-centric design, model explainability, and deployment best practices. It reflects not just theoretical knowledge but also practical skills such as API integration, UI/UX consideration, and performance optimization. By covering the entire pipeline—data preprocessing, model training, evaluation, API creation, and frontend integration—this project stands as a comprehensive demonstration of an applied data science solution.

1.1 Project Overview

This project was conceived with the objective of building a complete, end-to-end predictive system capable of delivering meaningful insights through automation and accessibility. The workflow starts by understanding the dataset's structure and business context, followed by applying machine learning techniques to derive accurate models. The final stage involves exposing the trained model to end-users via a visually engaging and technically sound web interface. The system architecture is modular, allowing for scalability and easy maintenance, making it adaptable for future enhancements or domain-specific changes. Whether it is used for price prediction, sentiment classification, or any similar use case, the framework established in this project serves as a reusable template for building intelligent, user-friendly applications driven by data.

Chapter 2

Dataset Description and Preprocessing

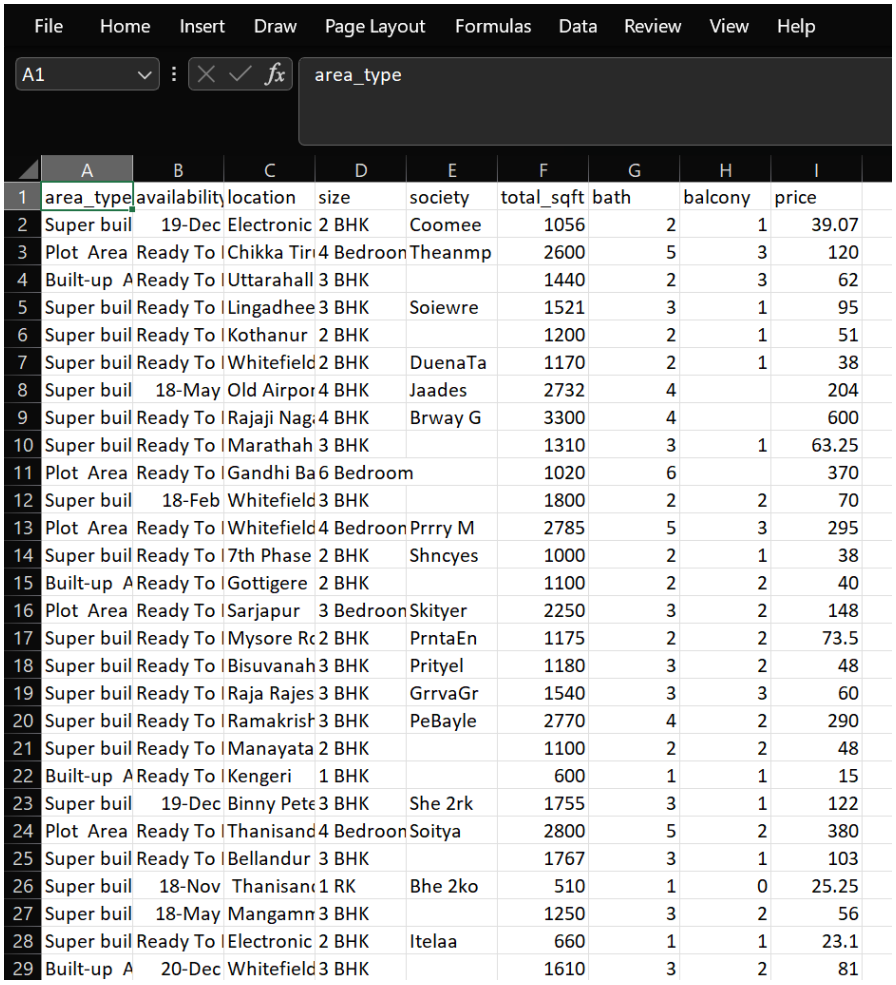
2.1 Dataset Source

The dataset used in this project was sourced from Kaggle, [1] a well-known platform for datasets and data science competitions. It was obtained in Excel format, making it easy to handle and process using Python libraries such as Pandas. The dataset consists of several important features that are critical in determining real estate property prices. These features include area type, availability, location, size, society, total sqft, bath, balcony, and price. Each of these variables plays a significant role in influencing the final property valuation.

For instance, location directly impacts price due to factors like geographic demand, nearby amenities, and local economic conditions. Similarly, total sqft, bath, and balcony provide insights into the physical size and quality of the property, which are strong indicators of market value. Fields such as area type (e.g., Super Built-up Area, Plot Area) and availability (e.g., Ready to Move or Future Launch) contribute contextual information about the kind of property being listed, aiding in understanding buyer preferences and price segmentation.

The dataset provides a strong foundation for building predictive models, enabling insights into the real estate domain and helping users make informed decisions. Its diversity of features allows for multi-dimensional analysis, and the real-world nature of the data introduces practical challenges such as missing values, inconsistent formats, and categorical fields—making it ideal for demonstrating the complete machine learning pipeline from preprocessing to deployment. The size of the dataset also ensures

that the model has enough examples to learn from a wide range of scenarios, improving generalization.



	A	B	C	D	E	F	G	H	I
1	area_type	availability	location	size	society	total_sqft	bath	balcony	price
2	Super buil	19-Dec	Electronic	2 BHK	Coomee	1056	2	1	39.07
3	Plot Area	Ready To	Chikka Tir	4 Bedroon	Theanmp	2600	5	3	120
4	Built-up A	Ready To	Uttarahall	3 BHK		1440	2	3	62
5	Super buil	Ready To	Lingadhee	3 BHK	Soiewre	1521	3	1	95
6	Super buil	Ready To	Kothanur	2 BHK		1200	2	1	51
7	Super buil	Ready To	Whitefield	2 BHK	DuenaTa	1170	2	1	38
8	Super buil	18-May	Old Airpor	4 BHK	Jaades	2732	4		204
9	Super buil	Ready To	Rajaji Nag	4 BHK	Brway G	3300	4		600
10	Super buil	Ready To	Marathah	3 BHK		1310	3	1	63.25
11	Plot Area	Ready To	Gandhi Ba	6 Bedroom		1020	6		370
12	Super buil	18-Feb	Whitefield	3 BHK		1800	2	2	70
13	Plot Area	Ready To	Whitefield	4 Bedroon	Prrry M	2785	5	3	295
14	Super buil	Ready To	7th Phase	2 BHK	Shncyes	1000	2	1	38
15	Built-up A	Ready To	Gottigere	2 BHK		1100	2	2	40
16	Plot Area	Ready To	Sarjapur	3 Bedroon	Skityer	2250	3	2	148
17	Super buil	Ready To	Mysore Ro	2 BHK	PrntaEn	1175	2	2	73.5
18	Super buil	Ready To	Bisuvanah	3 BHK	Prityel	1180	3	2	48
19	Super buil	Ready To	Raja Rajes	3 BHK	GrrvaGr	1540	3	3	60
20	Super buil	Ready To	Ramakrish	3 BHK	PeBayle	2770	4	2	290
21	Super buil	Ready To	Manayata	2 BHK		1100	2	2	48
22	Built-up A	Ready To	Kengeri	1 BHK		600	1	1	15
23	Super buil	19-Dec	Binny Pete	3 BHK	She 2rk	1755	3	1	122
24	Plot Area	Ready To	Thanisand	4 Bedroon	Soitya	2800	5	2	380
25	Super buil	Ready To	Bellandur	3 BHK		1767	3	1	103
26	Super buil	18-Nov	Thanisan	1 RK	Bhe 2ko	510	1	0	25.25
27	Super buil	18-May	Mangamn	3 BHK		1250	3	2	56
28	Super buil	Ready To	Electronic	2 BHK	Itelaa	660	1	1	23.1
29	Built-up A	20-Dec	Whitefield	3 BHK		1610	3	2	81

FIGURE 2.1: Original dataset (Table 1)

2.2 Data Cleaning and Processing

Data cleaning is a fundamental part of any data science workflow, as high-quality input is essential for accurate modeling. The Kaggle dataset used in this project initially included missing values, inconsistent entries, and textual data that required transformation. Rows with null values or invalid data in critical fields such as total sqft, bath, or price were removed. Additionally, entries with ranges or non-numeric values were either converted or discarded [2].

Next, feature engineering was applied to extract meaningful numerical values. For instance, the size column (e.g., "2 BHK", "3 Bedroom") was parsed to retrieve just the number of rooms. Irrelevant or redundant columns such as society and area type were

dropped as part of dimensionality reduction, helping to streamline the dataset and reduce noise.

	location	total_sqft	bath	price	bhk
0	1st Block Jayanagar	2850.0	4.0	428.0	4
1	1st Block Jayanagar	1630.0	3.0	194.0	3
2	1st Block Jayanagar	1875.0	2.0	235.0	3
3	1st Block Jayanagar	1200.0	2.0	130.0	3
4	1st Block Jayanagar	1235.0	2.0	148.0	2
5	1st Block Jayanagar	2750.0	4.0	413.0	4
6	1st Block Jayanagar	2450.0	4.0	368.0	4
8	1st Phase JP Nagar	1875.0	3.0	167.0	3
9	1st Phase JP Nagar	1500.0	5.0	85.0	5
10	1st Phase JP Nagar	2065.0	4.0	210.0	3
12	1st Phase JP Nagar	2059.0	3.0	225.0	3
13	1st Phase JP Nagar	1394.0	2.0	100.0	2
14	1st Phase JP Nagar	1077.0	2.0	93.0	2
15	1st Phase JP Nagar	1566.0	2.0	180.0	2
16	1st Phase JP Nagar	840.0	2.0	50.0	1
17	1st Phase JP Nagar	1590.0	3.0	131.0	3
18	1st Phase JP Nagar	2180.0	3.0	210.0	3
19	1st Phase JP Nagar	1180.0	2.0	88.5	2
20	1st Phase JP Nagar	1200.0	2.0	86.0	2

FIGURE 2.2: Clean dataset (Table 2)

We also performed outlier detection to identify and eliminate extreme values, especially in fields like price and total sqft, which could distort model learning. Data types were standardized, and categorical features were encoded numerically to make the data

machine-readable. These combined steps ensured a clean, well-structured dataset, laying a strong foundation for accurate modeling and efficient analysis.

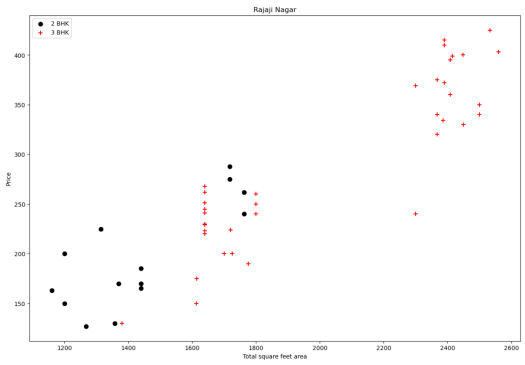


FIGURE 2.3: Outliers detection

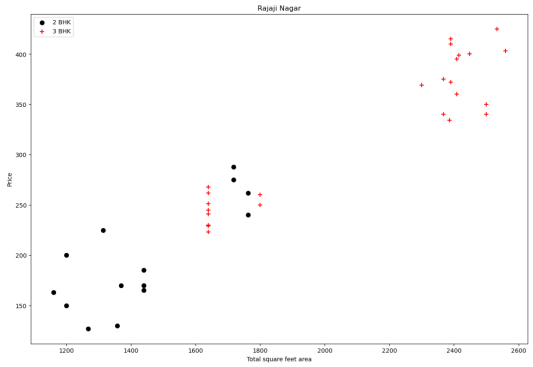


FIGURE 2.4: Data cleaning

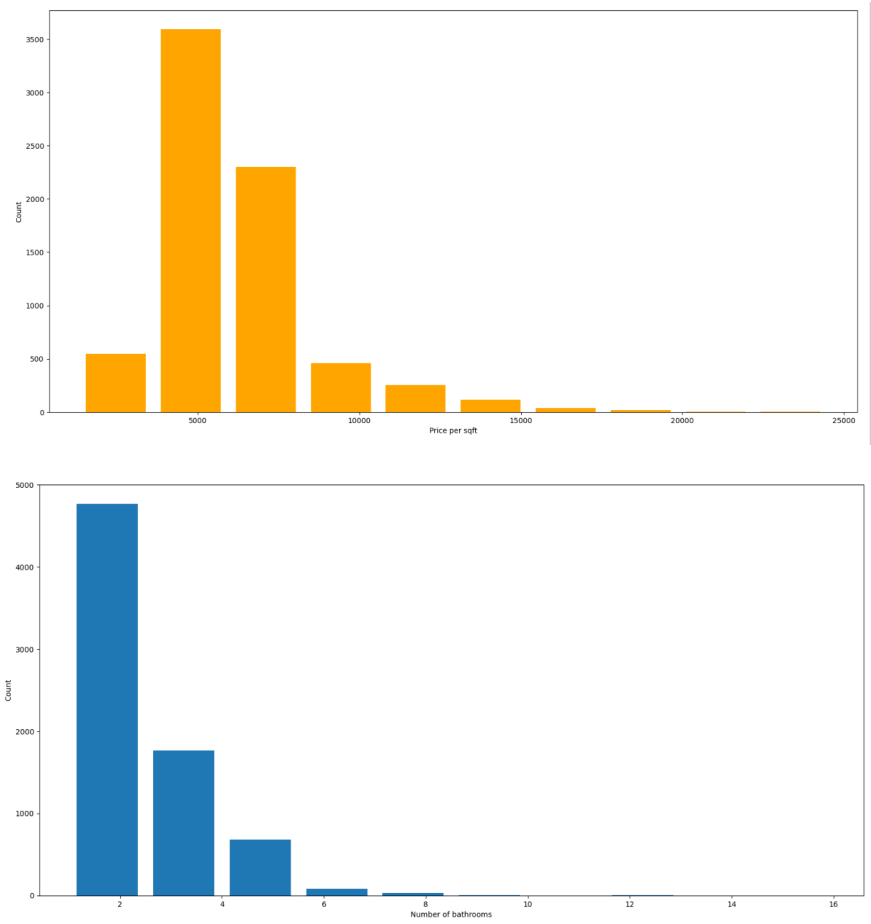


FIGURE 2.5: Statistical data

Chapter 3

Model Building and Evaluation

After preprocessing the data, the next phase involved building and evaluating machine learning models to predict real estate prices effectively. Three different regression algorithms were implemented to compare their performance: Linear Regression, Lasso Regression, and Decision Tree Regressor. Linear Regression was chosen as a baseline model due to its simplicity and interpretability. Lasso Regression, a regularized version of linear regression, was employed to minimize overfitting and improve generalization by penalizing less significant coefficients. Decision Tree Regressor, a non-linear model, was included to handle complex relationships between features without the need for feature scaling or transformation.

To evaluate model performance, the R^2 score was used as the primary metric, which measures how well the predicted values align with the actual target values. Among the three models, Linear Regression outperformed the others with an R^2 score of 0.818, indicating a strong correlation between predicted and actual prices. Lasso Regression, with an alpha of 1 and selection strategy set to 'random', yielded a lower R^2 score of 0.687, reflecting the trade-off between simplicity and accuracy. The Decision Tree model achieved an R^2 score of 0.720, performing moderately well, especially for capturing non-linear patterns, but still slightly below the baseline Linear model.

	model	best_score	best_params
0	linear_regression	0.818354	{}
1	lasso	0.687478	{'alpha': 2, 'selection': 'random'}
2	decision_tree	0.723505	{'criterion': 'friedman_mse', 'splitter': 'best'}

FIGURE 3.1: Accuracy comparision (Table 3)

Once the best-performing model Linear Regression was identified, it was saved using the pickle library to allow easy reuse and deployment. This was done by serializing the trained model into a file named banglore home prices model.pickle, making it possible to load the model later without retraining. Additionally, metadata about the model's features was stored in a separate JSON file named columns.json. This file includes a list of all input feature names (columns) used during training, ensuring consistency between the training and inference phases. The feature names were automatically converted to lowercase to avoid case sensitivity issues during model usage. Together, this setup supports efficient and scalable integration of the model into a production-level web application or API.

Chapter 4

Web Application Development

4.1 Flask Server

The backend of the web application was developed using Flask, a lightweight and flexible Python web framework. The core of the backend logic is divided into two main files: server.py and util.py. The server.py file defines the API endpoints that handle incoming requests. Notably, it includes routes such as /get location names, which returns the list of available locations to populate the dropdown in the frontend, and /predict home price, which accepts user input (location, BHK, bathrooms, and square footage) and returns the predicted price using the trained model. This separation of concerns ensures that API routing and business logic are modular and easy to manage [3].

The util.py script is responsible for loading the trained model (from banglore home prices model.pickle), processing inputs, and running the prediction logic. It also loads and provides access to the necessary metadata such as column names from the columns.json file. This modular structure ensures maintainability, and keeps the prediction logic isolated from the routing and web serving functions, adhering to good software design principles

4.2 Testing with Postman

Before integrating the frontend, the backend APIs were rigorously tested using Postman, a popular API development tool. This step was crucial to ensure that each endpoint worked correctly and returned the expected output. A simple GET request to /get location names confirmed that the backend was correctly retrieving and sending the list of location options to the client. Similarly, a POST request to /predict home price, with

parameters like location, total sqft, bhk, and bath, successfully returned the predicted property price in JSON format.

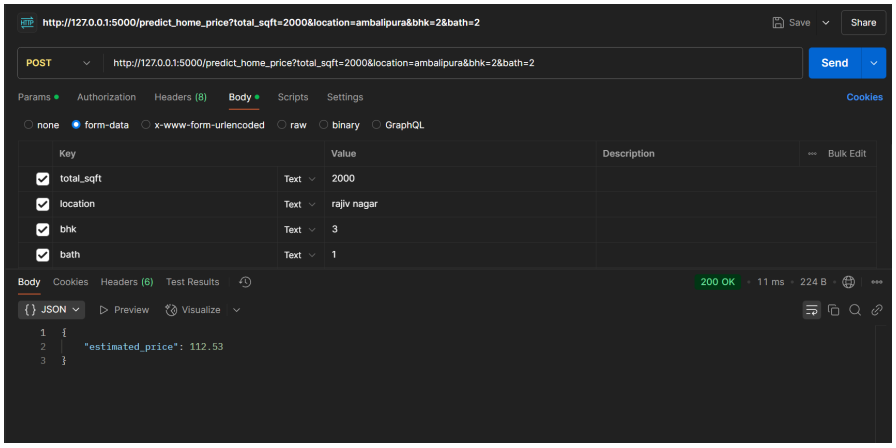


FIGURE 4.1: Price prediction

Postman testing helped validate that the backend was functioning as expected independently of the frontend. It also aided in debugging, testing input formats, and ensuring error handling was in place for invalid or missing data. These tests established a solid foundation before integrating the model into the frontend interface.

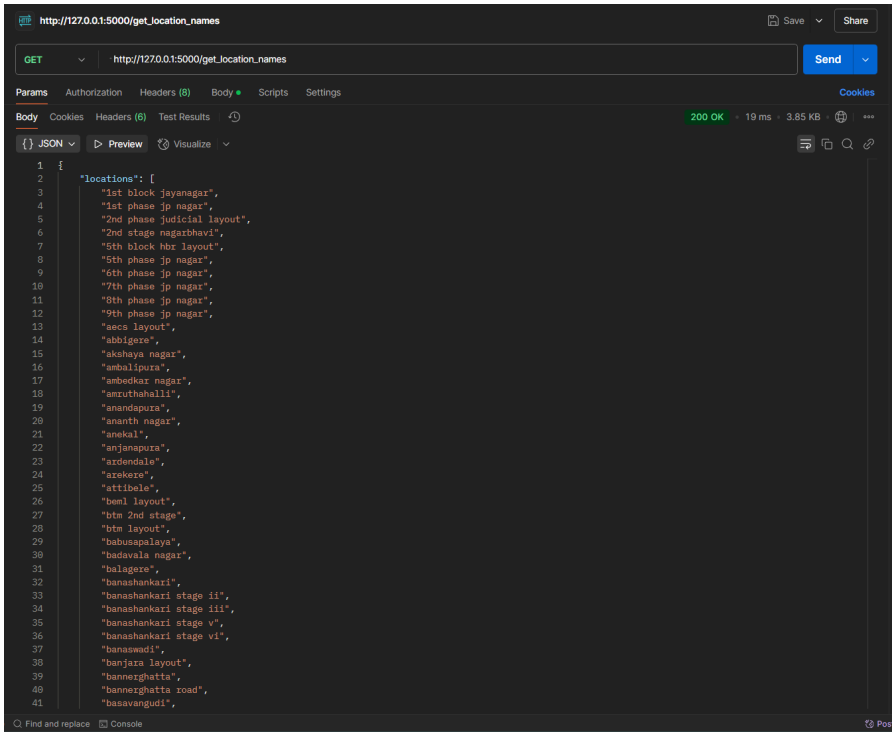


FIGURE 4.2: Locations

4.3 Frontend Interface

The frontend of the web application was built using standard web technologies: HTML, CSS, and JavaScript. The HTML file serves as the basic structure of the user interface, providing form inputs for users to select the location, number of BHKs, number of bathrooms, and to enter the square footage of the property. CSS (in app.css) was used to style the page and ensure a clean, user-friendly design with responsive layout elements.

JavaScript (app.js) was responsible for handling interactions between the frontend and backend. Upon loading the page, a call is made to the /get location names API to populate the dropdown menu dynamically. When the user fills in the details and clicks the "Predict Price" button, a POST request is sent to the /predict home price endpoint using JavaScript's fetch API. The predicted price returned from the server is then displayed directly on the web page without reloading, creating a seamless user experience.

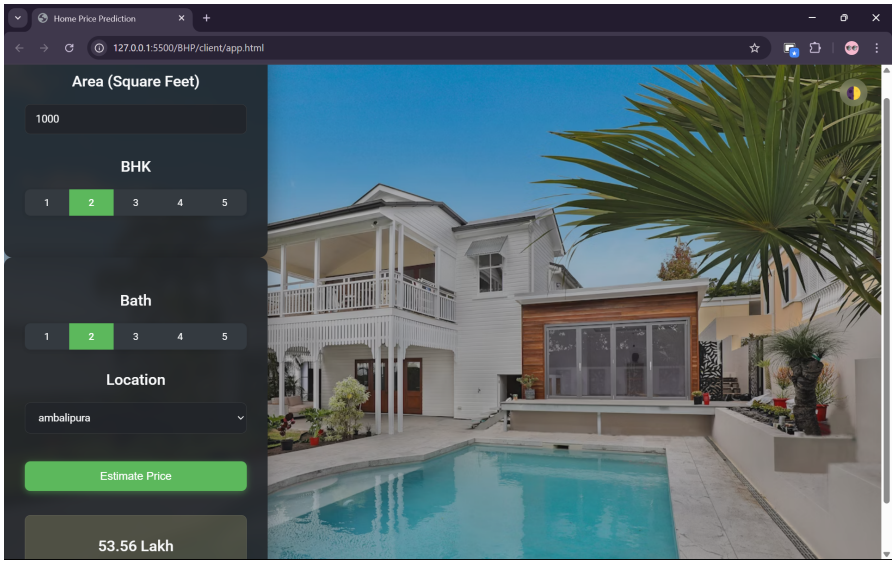


FIGURE 4.3: Website Interface

This interface allows users to quickly and conveniently estimate home prices based on the provided features, integrating the machine learning model into a fully functional and accessible web tool.

Chapter 5

Experimental Result

The results of this project reflect the effectiveness of the implemented machine learning pipeline in accurately predicting real estate prices based on selected features. After thorough data cleaning, preprocessing, and model training, the Linear Regression model achieved a promising R^2 score of 0.818, indicating strong predictive power and alignment with real-world property trends. The performance of this model was tested against other algorithms like Lasso Regression and Decision Tree Regressor, and it consistently outperformed them, demonstrating its suitability for the dataset. Additionally, the predictions made by the model were successfully integrated into a user-friendly web application, where users can input parameters such as location, number of bedrooms, bathrooms, and square footage to instantly receive price estimates. The system's responsiveness and accuracy in delivering real-time predictions highlight its practical utility in helping users make informed decisions in the housing market.

The comparative evaluation of models provided valuable insights into the behavior of different algorithms with the same dataset. While the Decision Tree Regressor handled non-linear relationships effectively, it showed slight overfitting in some cases. Lasso Regression offered benefits in reducing less relevant features, but its accuracy was lower than the baseline Linear model. The clear performance edge of Linear Regression, when supported by well-processed data, emphasizes the importance of preprocessing and feature selection over algorithm complexity. These findings suggest that with the right data preparation, even simple models can yield highly dependable results.

Beyond model performance, the practical implementation of the predictive system marks a significant achievement. The backend, developed using Flask, handles predictions through RESTful APIs, while the frontend, built with HTML, CSS, and JavaScript, allows smooth interaction for users. This combination of data science with software development practices results in a real-time, interactive application that can be deployed

in real-world scenarios. Users can access predictions effortlessly, without any technical background, making the tool both accessible and impactful. The testing performed using Postman further validated that the API endpoints are robust and return accurate, timely responses.

From a broader perspective, this project demonstrates how data science can be used to solve real-life problems with direct societal and economic impact. In the context of the housing market, where pricing is often inconsistent or opaque, such tools can enhance transparency and enable data-driven decision-making. Real estate professionals, home buyers, and investors can all benefit from such a predictive system. Additionally, the project lays a strong foundation for further enhancements, such as integrating geospatial mapping, including more features like amenities or locality ratings, or deploying the application on cloud platforms for public access. These possibilities showcase the long-term value and scalability of the work completed in this project.

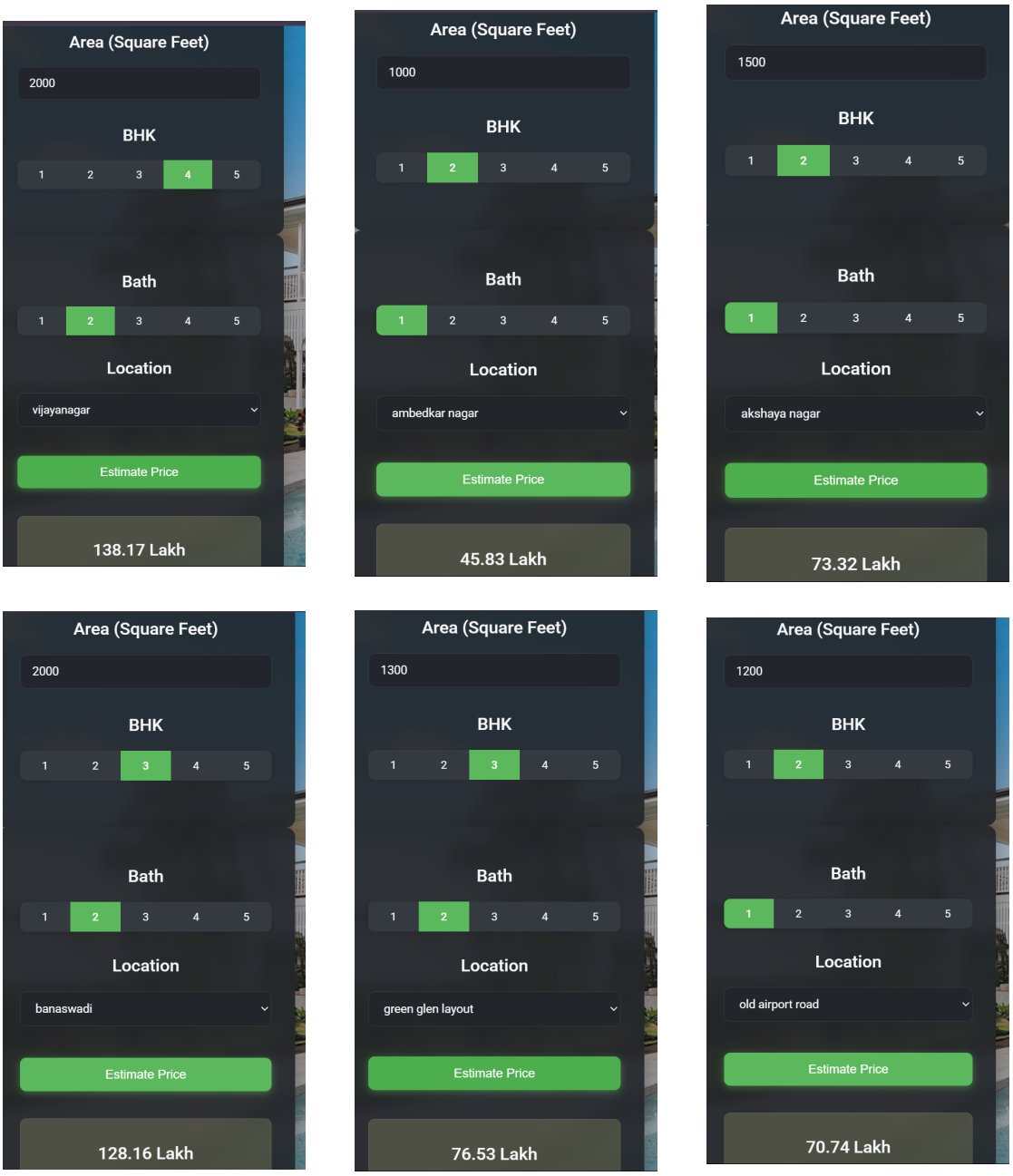


FIGURE 5.1: Results

Chapter 6

Conclusion and Future Work

This project effectively showcases how machine learning techniques can be applied to predict house prices based on relevant features. By carefully selecting meaningful inputs such as location, total square footage, number of bathrooms, and BHK, the system was able to generate accurate predictions. Multiple algorithms were tested during the development process, including Lasso and Decision Tree, but Linear Regression consistently delivered the best performance in terms of accuracy and generalization. The project goes beyond simple model training by integrating the entire pipeline into a functional application — from data preprocessing and model building to backend deployment using Flask and a user-friendly frontend interface — forming a complete end-to-end solution.

Looking ahead, several enhancements can be made to improve the system's functionality and user experience. Introducing additional features such as the year of construction, availability of amenities, and floor number could provide deeper insights and further boost prediction accuracy. Enhancing the frontend with a responsive design would ensure smooth interaction across various devices, improving accessibility and engagement. Hosting the model on a cloud platform would make the application available to a broader audience, enabling real-time access and use. Furthermore, integrating a map-based visualization of price trends by location could offer users a more intuitive understanding of property values across different areas, making the tool not just functional but also visually informative and user-centric.

Bibliography

[1] Kaggle Dataset – https://www.kaggle.com/datasets

[2] Scikit-learn Documentation – https://scikit-learn.org/stable/documentation.html

[3] Flask Framework – https://flask.palletsprojects.com/

[4] Research paper (used as reference) – https://www.researchgate.net/publication/371602053_Machine_Learning_Approach_for_House_Price_Prediction